

Chapter 2

Java Classes and Objects

Java is an Object-Oriented Language. As a language that has the Object Oriented feature, Java supports the following fundamental concepts:

- Polymorphism
- Inheritance
- Encapsulation
- Abstraction
- Classes
- Objects
- Instance
- Method
- Message Parsing

In this chapter, we will look into the concepts Classes and Objects.

•**Object** - Objects have states and behaviors. Example: A dog has states - color, name, breed as well as behaviors -wagging, barking, eating. An object is an instance of a class.

•**Class** - A class can be defined as a template/blue print that describes the behaviors/states that object of its type support.

Objects in Java:

Let us now look deep into what are objects. If we consider the real-world we can find many objects around us, Cars, Dogs, Humans, etc. All these objects have a state and behavior.

If we consider a dog, then its state is - name, breed, color, and the behavior is - barking, wagging, running

If you compare the software object with a real world object, they have very similar characteristics.

Software objects also have a state and behavior. A software object's state is stored in fields and behavior is shown via methods.

So in software development, methods operate on the internal state of an object and the object-to-object communication is done via methods.

Classes in Java:

A class is a blue print from which individual objects are created.

A sample of a class is given below:

```
public class Dog{  
    String breed;  
    int ageC  
    String color;  
  
    void barking(){  
    }  
  
    void hungry(){  
    }  
  
    void sleeping(){  
    }  
}
```

A class can contain any of the following variable types.

- Local variables:** Variables defined inside methods, constructors or blocks are called local variables. The variable will be declared and initialized within the method and the variable will be destroyed when the method has completed.
- Instance variables:** Instance variables are variables within a class but outside any method. These variables are initialized when the class is instantiated. Instance variables can be accessed from inside any method, constructor or blocks of that particular class.
- Class variables:** Class variables are variables declared with in a class, outside any method, with the static keyword.

A class can have any number of methods to access the value of various kinds of methods. In the above example, barking(), hungry() and sleeping() are methods.

Below mentioned are some of the important topics that need to be discussed when looking into classes of the Java Language.

Constructors:

When discussing about classes, one of the most important sub topic would be

constructors. Every class has a constructor. If we do not explicitly write a constructor for a class the Java compiler builds a default constructor for that class.

Each time a new object is created, at least one constructor will be invoked. The main rule of constructors is that they should have the same name as the class. A class can have more than one constructor.

Example of a constructor is given below:

```
public class Puppy{  
    public Puppy(){  
    }  
  
    public Puppy(String name){  
        // This constructor has one parameter, name.  
    }  
}
```

Java also supports Singleton Classes where you would be able to create only one instance of a class.

Note: We have two different types of constructors we are going to discuss constructors in detail in coming chapters.

Creating an Object:

As mentioned previously, a class provides the blueprints for objects. So basically an object is created from a class. In Java, the new key word is used to create new objects.

There are three steps when creating an object from a class:

- Declaration:** A variable declaration with a variable name with an object type.
- Instantiation:** The 'new' key word is used to create the object.
- Initialization:** The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

Example of creating an object is given below:

```
public class Puppy{  
  
    public Puppy(String name){  
        // This constructor has one parameter, name.  
        System.out.println("Passed Name is : " + name );  
    }  
}
```

```

    }

    public static void main(String []args){
        // Following statement would create an object myPuppy
        Puppy myPuppy = new Puppy( "tommy" );
    }
}

```

If we compile and run the above program, then it would produce the following result:

```
Passed Name is :tommy
```

Accessing Instance Variables and Methods:

Instance variables and methods are accessed via created objects. To access an instance variable the fully qualified path should be as follows:

```

/* First create an object */
ObjectReference = new Constructor();

/* Now call a variable as follows */
ObjectReference.variableName;

/* Now you can call a class method as follows */
ObjectReference.MethodName();

```

Example:

This example explains how to access instance variables and methods of a class:

```

public class Puppy{

    int puppyAge;

    public Puppy(String name){
        // This constructor has one parameter, name.
        System.out.println("Name chosen is :"+ name );
    }

    public void setAge( int age ){
        puppyAge = age;
    }
}

```

```

    }

    public int getAge( ){
        System.out.println("Puppy's age is : " + puppyAge );
        return puppyAge;
    }

    public static void main(String []args){
        /* Object creation */
        Puppy myPuppy = new Puppy( "tommy" );

        /* Call class method to set puppy's age */
        myPuppy.setAge( 2 );

        /* Call another class method to get puppy's age */
        myPuppy.getAge( );

        /* You can access instance variable as follows as well */
        System.out.println("Variable Value : " + myPuppy.puppyAge );
    }
}

```

If we compile and run the above program, then it would produce the following result:

```

Name chosen is :tommy
Puppy's age is :2
Variable Value :2

```

Source file declaration rules:

As the last part of this section let's now look into the source file declaration rules. These rules are essential when declaring classes, *import* statements and *package* statements in a source file.

- There can be only one public class per source file.
- A source file can have multiple non public classes.
- The public class name should be the name of the source file as well which should be appended by **.java** at the end. For example: the class name is *public class Employee{}* then the source file should be as Employee.java.
- If the class is defined inside a package, then the package statement should

be the first statement in the source file.

- If import statements are present then they must be written between the package statement and the class declaration. If there are no package statements then the import statement should be the first line in the source file.

- Import and package statements will imply to all the classes present in the source file. It is not possible to declare different import and/or package statements to different classes in the source file.

Classes have several access levels and there are different types of classes; abstract classes, final classes, etc. we will be explaining about all these in the access modifiers chapter.

Apart from the above mentioned types of classes, Java also has some special classes called Inner classes and Anonymous classes.

Java Package:

In simple, it is a way of categorizing the classes and interfaces. When developing applications in Java, hundreds of classes and interfaces will be written, therefore categorizing these classes is a must as well as makes life much easier.

Import statements:

In Java if a fully qualified name, which includes the package and the class name, is given then the compiler can easily locate the source code or classes. Import statement is a way of giving the proper location for the compiler to find that particular class.

For example, the following line would ask compiler to load all the classes available in directory `java_installation/java/io` :

```
import java.io.*;
```

A Simple Case Study

For our case study, we will be creating two classes. They are Employee and EmployeeTest.

First open notepad and add the following code. Remember this is the Employee class and the class is a public class. Now, save this source file with the name Employee.java.

The Employee class has four instance variables name, age, designation and salary. The class has one explicitly defined constructor, which takes a parameter.

```
import java.io.*;

public class Employee{

    String name;
    int age;
    String designation;
    double salary;

    // This is the constructor of the class Employee
    public Employee(String name){
        this.name = name;
    }

    // Assign the age of the Employee to the variable age.
    public void empAge(int empAge){
        age = empAge;
    }

    /* Assign the designation to the variable designation.*/
    public void empDesignation(String empDesig){
        designation = empDesig;
    }

    /* Assign the salary to the variable salary.*/
    public void empSalary(double empSalary){
        salary = empSalary;
    }

    /* Print the Employee details */
    public void printEmployee(){
        System.out.println("Name:" + name );
        System.out.println("Age:" + age );
        System.out.println("Designation:" + designation );
        System.out.println("Salary:" + salary);
    }
}
```

As mentioned previously in this tutorial, processing starts from the main method. Therefore in-order for us to run this Employee class there should be main method and objects should be created. We will be creating a separate class for these tasks.

Given below is the *EmployeeTest* class, which creates two instances of the class *Employee* and invokes the methods for each object to assign values for each variable.

Save the following code in *EmployeeTest.java* file

```
import java.io.*;

public class EmployeeTest{
    public static void main(String args[]){
        /* Create two objects using constructor */
        Employee empOne = new Employee("James Smith");
        Employee empTwo = new Employee("Mary Anne");
        // Invoking methods for each object created
        empOne.empAge(26);
        empOne.empDesignation("Senior Software Engineer");
        empOne.empSalary(1000);
        empOne.printEmployee();

        empTwo.empAge(21);
        empTwo.empDesignation("Software Engineer");
        empTwo.empSalary(500);
        empTwo.printEmployee();
    }
}
```

Now, compile both the classes and then run *EmployeeTest* to see the result as follows:

```
C:\> javac Employee.java
C:\> javac EmployeeTest.java
C:\> java EmployeeTest
Name:James Smith
Age:26
Designation:Senior Software Engineer
Salary:1000.0
Name:Mary Anne
Age:21
Designation:Software Engineer
Salary:500.0
```


Polymorphism

The dictionary definition of polymorphism refers to a principle in biology in which an organism or species can have many different forms or stages. This principle can also be applied to object-oriented programming and languages like the Java language. Subclasses of a class can define their own unique behaviors and yet share some of the same functionality of the parent class.

Polymorphism can be demonstrated with a minor modification to the Bicycle class. For example, a `printDescription` method could be added to the class that displays all the data currently stored in an instance.

```
public void printDescription(){
    System.out.println("\nBike is " + "in gear " + this.gear
        + " with a cadence of " + this.cadence +
        " and travelling at a speed of " + this.speed + ". ");
}
```

To demonstrate polymorphic features in the Java language, extend the Bicycle class with a MountainBike and a RoadBike class. For MountainBike, add a field for suspension, which is a String value that indicates if the bike has a front shock absorber, Front. Or, the bike has a front and back shock absorber, Dual.

Here is the updated class:

```
public class MountainBike extends Bicycle {
    private String suspension;

    public MountainBike(
        int startCadence,
        int startSpeed,
        int startGear,
        String suspensionType){
        super(startCadence,
            startSpeed,
            startGear);
        this.setSuspension(suspensionType);
    }

    public String getSuspension(){
        return this.suspension;
    }

    public void setSuspension(String suspensionType) {
        this.suspension = suspensionType;
    }

    public void printDescription() {
        super.printDescription();
        System.out.println("The " + "MountainBike has a" +
            getSuspension() + " suspension.");
    }
}
```

Note the overridden `printDescription` method. In addition to the information provided before, additional data about the suspension is included to the output.

Next, create the RoadBike class. Because road or racing bikes have skinny tires, add an attribute to track the tire width. Here is the RoadBike class:

```

public class RoadBike extends Bicycle{
    // In millimeters (mm)
    private int tireWidth;

    public RoadBike(int startCadence,
                    int startSpeed,
                    int startGear,
                    int newTireWidth){
        super(startCadence,
              startSpeed,
              startGear);
        this.setTireWidth(newTireWidth);
    }

    public int getTireWidth(){
        return this.tireWidth;
    }

    public void setTireWidth(int newTireWidth){
        this.tireWidth = newTireWidth;
    }

    public void printDescription(){
        super.printDescription();
        System.out.println("The RoadBike" + " has " + getTireWidth() +
                           " MM tires.");
    }
}

```

Note that once again, the printDescription method has been overridden. This time, information about the tire width is displayed.

To summarize, there are three classes: Bicycle, MountainBike, and RoadBike. The two subclasses override the printDescription method and print unique information.

Here is a test program that creates three Bicycle variables. Each variable is assigned to one of the three bicycle classes. Each variable is then printed.

```

public class TestBikes {
    public static void main(String[] args){
        Bicycle bike01, bike02, bike03;

        bike01 = new Bicycle(20, 10, 1);
        bike02 = new MountainBike(20, 10, 5, "Dual");
        bike03 = new RoadBike(40, 20, 8, 23);

        bike01.printDescription();
        bike02.printDescription();
        bike03.printDescription();
    }
}

```

The following is the output from the test program:

Bike is in gear 1 with a cadence of 20 and travelling at a speed of 10.

Bike is in gear 5 with a cadence of 20 and travelling at a speed of 10.

The MountainBike has a Dual suspension.

Bike is in gear 8 with a cadence of 40 and travelling at a speed of 20.

The RoadBike has 23 MM tires.

The Java virtual machine (JVM) calls the appropriate method for the object that is referred to in each variable. It does not call the method that is defined by the variable's type. This behavior is referred to as virtual method invocation and demonstrates an aspect of the important polymorphism features in the Java language.

Abstract Classes

A class that is declared using “abstract” keyword is known as abstract class. It may or may not include abstract methods which means in abstract class you can have concrete methods (methods with body) as well along with abstract methods (without an implementation, without braces, and followed by a semicolon). An abstract class can not be **instantiated** (you are not allowed to create **object** of Abstract class).

Abstract class declaration

Specifying **abstract keyword** before the class during declaration, makes it abstract. Have a look at below code:

```
// Declaration using abstract keyword
abstract class AbstractDemo{
    // Concrete method: body and braces
    public void myMethod(){
        //Statements here
    }

    // Abstract method: without body and braces
    abstract public void anotherMethod();
}
```

Since abstract class allows concrete methods as well, it does not provide 100% abstraction. You can say that it provides partial abstraction. **Interfaces** are used for 100% abstraction (full **abstraction**)

Remember two rules:

- 1) If the class is having few abstract methods and few concrete methods: declare it as abstract class.
- 2) If the class is having only abstract methods: declare it as interface.

Error!! - Object creation of abstract class is not allowed

As discussed above, we cannot instantiate an abstract class. The following code throws an error.

```
abstract public class AbstractDemo{
    public void myMethod(){
        System.out.println("Hello");
    }
}
```

```

    abstract public void anotherMethod();
}
public class ConcreteDemo{

    public void anotherMethod() {
        System.out.print("Abstract method");
    }
    public static void main(String args[])
    {
        //Can't create object of abstract class - error!
        AbstractDemo obj = new AbstractDemo();
        obj.display();
    }
}

```

Output:

Unresolved compilation problem: Cannot instantiate the type AbstractEx1

Note: The class that extends the abstract class, have to implement all the abstract methods of abstract class, else they can be declared abstract in the class as well.

Why we need an abstract class?

Let me explain this with an example. Suppose there is a class `Animal` and there are few other classes like `Cat`, `Dog` and `Horse`. These classes extends `Animal` class so basically they are having few common habits(methods in technically) which they are inheriting from `Animal` class. Now, if you have understood the above example then you would have been able to figure out that **creating object of Animal class has no significance** as you can't judge that the **new** object of `Animal` class will represent which animal. Hence for such kind of scenarios we generally creates an **abstract class** and later **concrete classes** extends these classes and overrides their methods accordingly and can have their own methods as well.

Abstract vs Concrete

A class which is not abstract is referred as **Concrete class**. In the above example which I explained - `Animal` is a abstract class and `Cat`, `Dog` and `Horse` are concrete classes.

Key Points:

1. An abstract class has no use until unless it is extended by some other class.

2.If you declare an **abstract method** (discussed below) in a class then you must declare the class abstract as well. you can't have abstract method in a **non-abstract class**. It's vice versa is not always true: If a class is not having any abstract method then also it can be marked as abstract.

3.Abstract class can have non-abstract method (concrete) as well.

You may also like to read:

Difference between abstract class and Interface in Java

Abstract methods

Well, we already discussed about abstract methods in the above section. Lets take few examples to understand it better.

syntax:

```
public abstract void display();
```

Points to remember about abstract method:

- 1) Abstract method has no body.
- 2) Always end the declaration with a **semicolon(;)**.
- 3) It must be **overridden**. An abstract class must be extended and in a same way abstract method must be overridden.
- 4) Abstract method must be in a abstract class.

Note: The class which is extending abstract class must override (or implement) all the abstract methods.

Example of Abstract class and method

```
abstract class Demo1{
    public void disp1(){
        System.out.println("Concrete method of abstract class");
    }
    abstract public void disp2();
}

class Demo2 extends Demo1{
    /* I have given the body to abstract method of Demo1 class
    It is must if you don't declare abstract method of super class
    compiler would throw an error*/
    public void disp2()
    {
        System.out.println("I'm overriding abstract method");
    }
}
```

```
}  
public static void main(String args[]){  
    Demo2 obj = new Demo2();  
    obj.disp2();  
}  
}
```

Output:

I'm overriding abstract method

Interface in java with example programs

BY CHAITANYA SINGH | FILED UNDER: **OOPS CONCEPT**

Earlier we discussed about **abstract class** which is used to achieve partial abstraction(hiding irrelevant details from user). In this tutorial we are going to discuss about interfaces, which are used for achieving full abstraction. In this post, we will discuss what is an interface? what is the significance of it? when and how to use it?

Read: [Difference between abstract class and interface](#)

What is an interface?

Interface looks like class but it is not a class. An interface can have methods and variables just like the class but the methods declared in interface are by default abstract (only method signature, no body). Also, the variables declared in an interface are public, static & final by default. We will discuss these points in detail, later in this post.

What is the use of interfaces?

As mentioned above they are used for abstraction. Since methods in interfaces do not have body, they have to be implemented by the class before you can access them. The class that implements interface must implement all the methods of that interface. Also, java programming language does not support multiple inheritance, using interfaces we can achieve this as a class can implement more than one interfaces, however it cannot extend more than one classes.

Declaration

Interfaces are declared by specifying a keyword “interface”. E.g.:

```
interface MyInterface
```

```

{
    /* All the methods are public abstract by default
    * Note down that these methods are not having body
    */
    public void method1();
    public void method2();
}

```

Interface Implementation

This is how a class implements an interface. It has to provide the body of all the methods that are declared in interface.

Note: class **implements** interface but an interface **extends** another interface.

```

interface MyInterface
{
    public void method1();
    public void method2();
}
class XYZ implements MyInterface
{
    public void method1()
    {
        System.out.println("implementation of method1");
    }
    public void method2()
    {
        System.out.println("implementation of method2");
    }
    public static void main(String arg[])
    {
        MyInterface obj = new XYZ();
        obj.method1();
    }
}

```

Output:

```

implementation of method1

```

Interface and Inheritance

As discussed above, an interface can not implement another interface. It has to extend the other interface if required. See the below example where we have two interfaces Inf1 and Inf2. Inf2 extends Inf1 so If class implements the Inf2 it has to provide implementation of all the methods of interfaces Inf1 and Inf2.

```

public interface Inf1 {
    public void method1();
}
public interface Inf2 extends Inf1 {
    public void method2();
}

```

```
public class Demo implements Inf2{  
    public void method1(){  
        //Implementation of method1  
    }  
    public void method2(){  
        //Implementation of method2  
    }  
}
```

In the program above, “Demo” class is implementing only one interface “Inf2” however it has to provide the implementation of all the methods of interface “Inf1” too, because interface Inf2 extends Inf1.

Tag or Marker interface

An empty interface is known as tag or marker interface. For example Serializable, ActionListener, Remote(java.rmi.Remote) are tag interfaces. These interfaces do not have any field and methods in it. Read more about it [here](#).

Nested interfaces

An interface which is declared inside another interface or class is called nested interface. They are also known as inner interface. For example Entry interface in collections framework is declared inside Map interface, that’s why we don’t use it directly, rather we use it like this: Map.Entry. Read more about nested interface along with examples at the link: [Nested Interface with examples](#).

Key points: Here are the key points to remember about interfaces:

- 1) We can’t instantiate an interface in java.
- 2) Interface provides complete **abstraction** as none of its methods can have body. On the other hand, **abstract class** provides partial abstraction as it can have abstract and concrete(methods with body) methods both.
- 3) `implements` keyword is used by classes to implement an interface.
- 4) While providing implementation in class of any method of an interface, it needs to be mentioned as public.
- 5) Class implementing any interface must implement all the methods, otherwise the class should be declared as “abstract”.
- 6) Interface cannot be declared as private, protected or transient.

7) All the interface methods are by default **abstract and public**.

8) Variables declared in interface are **public, static and final** by default.

```
interface Try
{
    int a=10;
    public int a=10;
    public static final int a=10;
    final int a=10;
    static int a=0;
}
```

All of the above statements are identical.

9) Interface variables must be initialized at the time of declaration otherwise compiler will through an error.

```
interface Try
{
    int x;//Compile-time error
}
```

Above code will throw a compile time error as the value of the variable x is not initialized at the time of declaration.

10) Inside any implementation class, you cannot change the variables declared in interface because by default, they are public, static and final. Here we are implementing the interface “Try” which has a variable x. When we tried to set the value for variable x we got compilation error as the variable x is public static **final** by default and final variables can not be re-initialized.

```
class Sample implements Try
{
    public static void main(String args[])
    {
        x=20; //compile time error
    }
}
```

11) Any interface can extend any interface but cannot implement it. Class implements interface and interface extends interface.

12) A **class** can implement any **number of interfaces**.

13) If there are **two or more same methods** in two interfaces and a class implements both interfaces, implementation of the method once is enough.

```

interface A
{
    public void aaa();
}
interface B
{
    public void aaa();
}
class Central implements A,B
{
    public void aaa()
    {
        //Any Code here
    }
    public static void main(String args[])
    {
        //Statements
    }
}

```

14) A class cannot implement two interfaces that have methods with same name but different return type.

```

interface A
{
    public void aaa();
}
interface B
{
    public int aaa();
}
class Central implements A,B
{
    public void aaa() // error
    {
    }
    public int aaa() // error
    {
    }
    public static void main(String args[])
    {
    }
}

```

15) Variable names conflicts can be resolved by interface name e.g:

```

interface A
{
    int x=10;
}
interface B
{
    int x=100;
}
class Hello implements A,B
{

```

```

public static void Main(String args[])
{
    // reference to x is ambiguous both variables are x
    System.out.println(x);
    System.out.println(A.x);
    System.out.println(B.x);
}
}

```

Benefits of having interfaces:

Following are the advantages of using interfaces:

1. Without bothering about the implementation part, we can achieve the security of implementation
2. In java, **multiple inheritance** is not allowed, However by using interfaces you can achieve the same . A class can extend only one class but can implement any number of interfaces. It saves you from Deadly Diamond of Death(DDD) problem.

Difference between Abstract Class and Interface in Java with examples.

abstract Classes	Interfaces
1 abstract class can extend only one class or one abstract class at a time	interface can extend any number of interfaces at a time
2 abstract class can extend from a class or from an abstract class	interface can extend only from an interface
3 abstract class can have both abstract and concrete methods	interface can have only abstract methods
4 A class can extend only one abstract class	A class can implement any number of interfaces
5 In abstract class keyword 'abstract' is mandatory to declare a method as an abstract	In an interface keyword 'abstract' is optional to declare a method as an abstract
6 abstract class can have protected , public and public abstract methods	Interface can have only public abstract methods i.e. by default
7 abstract class can have static, final or static final variable with any access specifier	interface can have only static final (constant) variable i.e. by default

Each of the above difference between Abstract class vs Interface is explained with an example below -

Abstract class vs interface

Difference No.1:

Abstract class can extend only one class or one abstract class at a time

```

class Example1{
    public void display1(){
        System.out.println("display1 method");
    }
}

abstract class Example2{
    public void display2(){
        System.out.println("display2 method");
    }
}

abstract class Example3 extends Example1{
    abstract void display3();
}

class Example4 extends Example2{
    public void display3(){
        System.out.println("display3 method");
    }
}

class Demo{
    public static void main(String args[]){
        Example4 obj=new Example4();
        obj.display3();
    }
}

```

Output:

display3 method

Interface can extend any number of interfaces at a time

//first interface

```

interface Example1{
    public void display1();
}

```

//second interface

```

interface Example2 {
    public void display2();
}

```

//This interface is extending both the above interfaces

```

interface Example3 extends Example1,Example2{
}
class Example4 implements Example3{
    public void display1(){
        System.out.println("display2 method");
    }
    public void display2(){
        System.out.println("display3 method");
    }
}
class Demo{
    public static void main(String args[]){
        Example4 obj=new Example4();
        obj.display1();
    }
}

```

Output:

display2 method

Difference No.2:

Abstract class can be inherited by a class or an abstract class

```

class Example1{
    public void display1(){
        System.out.println("display1 method");
    }
}
abstract class Example2{
    public void display2(){
        System.out.println("display2 method");
    }
}
abstract class Example3 extends Example2{
    abstract void display3();
}
class Example4 extends Example3{
    public void display2(){
        System.out.println("Example4-display2 method");
    }
}

```

```

    }
    public void display3(){
        System.out.println("display3 method");
    }
}
class Demo{
    public static void main(String args[]){
        Example4 obj=new Example4();
        obj.display2();
    }
}

```

Output:

Example4-display2 method

Interfaces can be extended only by interfaces. Classes has to implement them instead of extend

```

interface Example1{
    public void display1();
}
interface Example2 extends Example1{
}
class Example3 implements Example2{
    public void display1(){
        System.out.println("display1 method");
    }
}
class Demo{
    public static void main(String args[]){
        Example3 obj=new Example3();
        obj.display1();
    }
}

```

Output:

display1 method

Difference No.3

Abstract class can have both abstract and concrete methods

```

abstract class Example1 {
    abstract void display1();
    public void display2(){
        System.out.println("display2 method");
    }
}

class Example2 extends Example1{
    public void display1(){
        System.out.println("display1 method");
    }
}

class Demo{
    public static void main(String args[]){
        Example2 obj=new Example2();
        obj.display1();
    }
}

```

Interface can only have abstract methods, they cannot have concrete methods

```

interface Example1{
    public abstract void display1();
}

class Example2 implements Example1{
    public void display1(){
        System.out.println("display1 method");
    }
}

class Demo{
    public static void main(String args[]){
        Example2 obj=new Example2();
        obj.display1();
    }
}

```

Difference No.4

A class can extend only one abstract class at a time

```

abstract class Example1{
    public void display1(){

```

```

        System.out.println("display1 method");
    }
}
abstract class Example2{
    abstract void display2();
}
class Example3 extends Example1{
    public void display3(){
        System.out.println("display3 method");
    }
}
class Demo{
    public static void main(String args[]){
        Example3 obj=new Example3();
        obj.display3();
    }
}

```

A class can implement any number of interfaces at a time

```

interface Example1{
    public void display1();
}
interface Example2{
    public void display2();
}
class Example3 implements Example1,Example2{
    public void display1(){
        System.out.println("display1 method");
    }
    public void display2(){
        System.out.println("display2 method");
    }
    public void display3(){
        System.out.println("display3 method");
    }
}
class Demo{
    public static void main(String args[]){

```



```

        Example3 obj=new Example3();
        obj.display1();
        obj.display3();
    }
}

```

Difference No.5

In abstract class, the keyword 'abstract' is mandatory to declare a method as an abstract

```

abstract class Example1{
    public abstract void display1();
}

class Example2 extends Example1{
    public void display1(){
        System.out.println("display1 method");
    }
    public void display2(){
        System.out.println("display2 method");
    }
}

class Demo{
    public static void main(String args[]){
        Example2 obj=new Example2();
        obj.display1();
    }
}

```

In interfaces, the keyword 'abstract' is optional to declare a method as an abstract because all the methods are abstract by default

```

interface Example1{
    public void display1();
}

class Example2 implements Example1{
    public void display1(){
        System.out.println("display1 method");
    }
    public void display2(){
        System.out.println("display2 method");
    }
}

```

```

    }
}
class Demo{
    public static void main(String args[]){
        Example2 obj=new Example2();
        obj.display1();
    }
}

```

Difference No.6

Abstract class can have protected , public and public abstract methods

```

abstract class Example1{
    protected abstract void display1();
    public abstract void display2();
    public abstract void display3();
}
class Example2 extends Example1{
    public void display1(){
        System.out.println("display1 method");
    }
    public void display2(){
        System.out.println("display2 method");
    }
    public void display3(){
        System.out.println("display3 method");
    }
}
class Demo{
    public static void main(String args[]){
        Example2 obj=new Example2();
        obj.display1();
    }
}

```

Interface can have only public abstract methods i.e. by default

```

interface Example1{
    void display1();
}
class Example2 implements Example1{

```

```

    public void display1(){
        System.out.println("display1 method");
    }
    public void display2(){
        System.out.println("display2 method");
    }
}
class Demo{
    public static void main(String args[]){
        Example2 obj=new Example2();
        obj.display1();
    }
}

```

Difference No.7

Abstract class can have static, final or static final variables with any access specifier

```

abstract class Example1{
    private int numOne=10;
    protected final int numTwo=20;
    public static final int numThree=500;
    public void display1(){
        System.out.println("Num1="+numOne);
    }
}
class Example2 extends Example1{
    public void display2(){
        System.out.println("Num2="+numTwo);
        System.out.println("Num2="+numThree);
    }
}
class Demo{
    public static void main(String args[]){
        Example2 obj=new Example2();
        obj.display1();
        obj.display2();
    }
}

```

Interface can have only static final (constant) variable i.e. by default

```
interface Example1{
    int numOne=10;
}

class Example2 implements Example1{
    public void display1(){
        System.out.println("Num1="+numOne);
    }
}

class Demo{
    public static void main(String args[]){
        Example2 obj=new Example2();
        obj.display1();
    }
}
```